



UNIVERSIDADE DO VALE DO ITAJAÍ
CIÊNCIA DA COMPUTAÇÃO – SISTEMAS OPERACIONAIS
PROF. MSc. FELIPE VIEL

GUSTAVO HENRIQUE STAHL MÜLLER

IMPLEMENTAÇÃO DE ALGORITMOS DE ESCALONAMENTO EM C

ITAJAÍ
2021

SUMÁRIO

1. INTRODUÇÃO.....	3
2. DESENVOLVIMENTO.....	4
2.1. ALGORITMO ROUND ROBIN.....	4
2.2. ALGORITMO ROUND ROBIN COM PRIORIDADE.....	6
3. CONCLUSÃO.....	7

1. INTRODUÇÃO

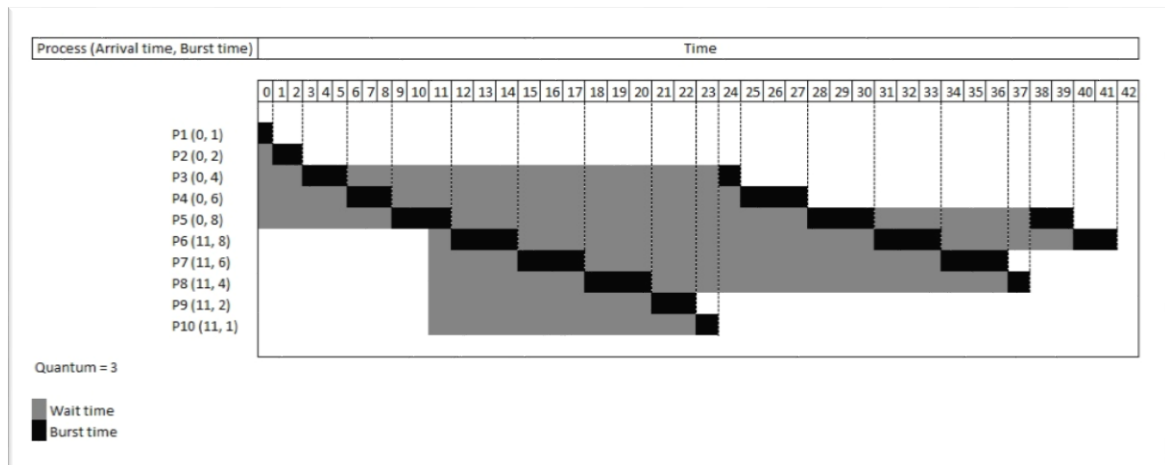
Esse trabalho demonstra o funcionamento e trechos relevantes do código-fonte da minha implementação de dois algoritmos de escalonamento em C: Round Robin e Round Robin com prioridade.

Link da apresentação: https://youtu.be/X66fCTc_-y8

2. DESENVOLVIMENTO

Algoritmos de escalonamento são de extrema importância para a Ciência da Computação. Eles são responsáveis por decidir a ordem em que as tarefas existentes irão ser executadas pelo processador. Além disso, são responsáveis pela realização de trocas de contexto, o que permite a execução concorrente de tarefas.

Figura 1 – Escalonamento de tarefas através do algoritmo Round Robin.



Fonte: Wikipédia.

2.1. ALGORITMO ROUND ROBIN

Round Robin é um algoritmo de escalonamento simples, que não aplica nenhuma prioridade específica à uma tarefa. Ou seja, todas as tarefas possuem a mesma prioridade e recebem o mesmo tempo de execução do processador. Na minha implementação, a ordem de execução começa pelos processos que foram adicionados à fila de aptos (variável *TASKS_HEAD*) primeiro.

No começo do algoritmo, o primeiro índice será 0, e ele será incrementado cada vez que o *quantum* de uma tarefa acabar. O nodo nesse índice é pego, e então executado pela função *run()*, que retornará 0 caso o processo não tenha chegado ao fim (caso o *burst* seja maior que 0) ou 1 caso tenha chegado ao fim (caso o *burst* seja igual à 0).

Figura 2 – Escalonamento de tarefas através do algoritmo Round Robin.

```
int currIndex = 0;
int size = getSize(TASKS_HEAD);

while(1) {
    ListNode* currNode = getAt(TASKS_HEAD, currIndex);

    // Runs and adds to burst
    int completed = run(currNode->task);
```

Fonte: O autor.

Caso o processo tenha terminado, ele será deletado da fila de tarefas do escalonador.

Figura 3 – Tarefa sendo deletada após término.

```
// Process has ended maximum burst: delete
if(completed == 1) {
    printf("Deleted task = [%s]\n", currNode->task->name);
    delete(&TASKS_HEAD, currNode->task);
    size--;
    currIndex--;
}
```

Fonte: O autor.

Por último, caso ainda tenha processos restantes, o índice de tarefa será incrementado. Se não, a execução de processos terminará.

Figura 4 – Checagem de existência de outras tarefas.

```
// Break if no elements left
if(size > 0) {
    currIndex = (currIndex + 1) % size;

    // Waits when re-executing list
    if(currIndex == 0) {
        Sleep(1000);
        system("@cls||clear");
    }
} else {
    break;
}
```

Fonte: O autor.

As tarefas usadas para fins de testes estão armazenadas no arquivo *rr-schedule.txt*. Cada linha é separada em três colunas: A primeira coluna é o nome da tarefa, a segunda é a sua prioridade (que no caso do Round Robin, é ignorada) e a terceira é o *burst* (o tempo de execução necessário para ser completada).

Figura 5 – Arquivo *rr-schedule.txt*.

T1, 40, 50
T2, 40, 50
T3, 40, 50
T4, 40, 50
T5, 40, 50
T6, 40, 50

Fonte: O autor.

No primeiro ciclo de execução, cada tarefa é executada por 15 unidades de tempo, que é o *quantum* definido pelo processador. Esse ciclo de execução se repete mais algumas vezes, até que as tarefas possuam um burst de somente 5 unidades de tempo, o que significa que na próxima vez que forem executadas, elas serão terminadas pelo escalonador.

Figura 6 – Primeiro ciclo de execução do Round Robin.

```
Running task = [T1] [40] [35/50] for 15 units.
Running task = [T2] [40] [35/50] for 15 units.
Running task = [T3] [40] [35/50] for 15 units.
Running task = [T4] [40] [35/50] for 15 units.
Running task = [T5] [40] [35/50] for 15 units.
Running task = [T6] [40] [35/50] for 15 units.
```

Fonte: O autor.

Na figura abaixo, a tarefa T2 possui somente 5 unidades de tempo restantes para execução. Após executar por 5 unidades de tempo, seu *burst* é igual à 0, o que implica na sua exclusão:

Figura 7 – Tarefa sendo deletada após seu *burst* chegar à 0.

```
Running task = [T2] [40] [0/50] for 5 units.
Deleted task = [T2]
```

Fonte: O autor.

2.2. ALGORITMO ROUND ROBIN COM PRIORIDADE

Round Robin com prioridade busca sempre a execução da tarefa com maior prioridade. Na minha implementação, essa execução ocorre sem preempção: ou seja, a tarefa de maior prioridade é executada integralmente, em contraste ao Round Robin simples, que executa as tarefas de maneira concorrente.

A primeira etapa do algoritmo é escolher a tarefa à ser executada; A prioridade somente será importante caso mais de uma tarefa exista. Nesse caso, a tarefa com maior prioridade será escolhida, através da função *getTaskWithMostPriority()*.

Figura 8 – Escolhendo a tarefa à ser executada no Round Robin com prioridade.

```
int size = getSize(TASKS_HEAD);

while(size > 0) {
    ListNode* currNode;

    // Only check priority if more than one task is present
    if(size > 1) {
        currNode = getTaskWithMostPriority(TASKS_HEAD);
    } else {
        currNode = getAt(TASKS_HEAD, 0);
    }
}
```

Fonte: O autor.

A próxima etapa é executar a tarefa até ela acabar, e então deletar ela da lista de tarefas. Isso irá se repetir até que todas as tarefas sejam concluídas.

Figura 9 – Executando uma tarefa e deletando ela após ser concluída.

```
// Executes until completion
while(run(currNode->task) == 0);

// Deletes executed process
printf("Deleted task = [%s]\n", currNode->task->name);
delete(&TASKS_HEAD, currNode->task);
size--;

// Waits when re-executing list
Sleep(1000);
system("@cls||clear");
```

Fonte: O autor.

O arquivo que contém as tarefas utilizadas para teste se chama *rr-schedule_pri.txt*, e possui a mesma estrutura que o arquivo *rr-schedule.txt* mencionado anteriormente.

Figura 10 – Arquivo *rr-schedule_pri.txt*.

T1, 40, 50
T2, 40, 50
T3, 40, 50
T4, 40, 50
T5, 40, 50
T6, 40, 50

Fonte: O autor.

3. CONCLUSÃO

Round Robin sem prioridade é um algoritmo de escalonamento de tarefas simples que não implica em inanição de tarefas e permite concorrência.

Round Robin com prioridade é mais complexo do que a sua versão sem prioridade, mas possui vantagens, como a execução adiada de tarefas prioritárias, mas também desvantagens, como a possível inanição de tarefas com pouca prioridade.